

Proyecto N°2

Profesor: Shaharyar Kamal

Auxiliares: Rodrigo Navarro, Gabriel Chávez

Ayudantes: Camilo Carreño, Agustín Gonzalez, Sebastian Carrasco

Debe entregar un reporte detallando su código y sus resultados. Además debe subir su presentación final del proyecto. Grupos de máximo 4 personas.

Introducción

En este proyecto, los estudiantes deberán diseñar e implementar versiones funcionales de distintos juegos clásicos —como Snake, el dinosaurio sin conexión de Google, Buscaminas, Space Invaders, Pac-Man y Connect 4— aplicando de forma integrada los principios de programación dinámica y programación orientada a objetos (POO).

El objetivo principal es que desarrollen soluciones que combinen eficiencia computacional y buen diseño modular, utilizando clases, herencia y encapsulamiento para estructurar el código, junto con técnicas de programación dinámica que optimicen el cálculo de estados, puntajes o trayectorias dentro del juego.

La programación dinámica resulta especialmente útil en este contexto porque permite almacenar y reutilizar resultados intermedios de cálculos que se repiten con frecuencia durante la ejecución del juego. Por ejemplo, puede emplearse para determinar caminos óptimos o movimientos mínimos (como en el desplazamiento del personaje o de los enemigos), calcular estrategias de búsqueda o evasión, optimizar la detección de colisiones en mapas complejos o actualizar puntajes y niveles de dificultad sin recalcular todo el historial. Gracias a esta técnica, los juegos pueden mantener un rendimiento fluido incluso cuando el número de elementos en pantalla crece, evitando operaciones redundantes y mejorando la experiencia del jugador.

Por su parte, la programación orientada a objetos permitirá organizar el código en torno a las entidades principales del juego —jugadores, enemigos, proyectiles, tableros o interfaces—, favoreciendo la reutilización, extensibilidad y claridad del diseño. De esta forma, cada componente podrá evolucionar de manera independiente, lo que facilita la incorporación de nuevas mecánicas o la depuración de errores sin alterar la estructura general del programa.

Descripción

Para la realización de este proyecto, su grupo debe inscribir uno de los siguientes juegos para desarrollar utilizando programación orientada a objetos (POO) y programación dinámica (DP), cuya inscripción será dentro de una planilla publicada en U-Cursos. **Importante:** En los siguientes puntos se describen puntos básicos de la implementación, pero se da libertad a que como grupo se decida cuales características desarrollar y la manera de implementar cada juego.

- **Snake:** El jugador controla una serpiente que se desplaza por el tablero, creciendo en largo al comer comida y perdiendo si choca consigo misma o los bordes del tablero. Se espera que mediante POO se modelen las distintas entidades del juego como serpiente y tablero mediante clases con sus propios atributos y métodos (posición, dirección, actualización del movimiento, actualización del largo, detección de colisiones). Para esto, se recomienda utilizar DP con el fin de almacenar posiciones visitadas o trayectorias óptimas al anticipar movimientos, evitando recalcular posibles colisiones.
- **Dinosaurio de Google** El personaje principal debe saltar o agacharse para esquivar obstáculos que aparecen con velocidad creciente, acumulando puntaje mientras más tiempo sobreviva. Los obstáculos en altura son 3: alta (Pterodactylus), media (cactus alto), baja (arbusto de cactus). Para los cactus existen 2 distancias: larga (grupo de cactus) y corta (1 cactus). Se debe implementar clases para jugador, obstáculos, entorno, sistema de puntaje actualizados y detección de eventos (salto, colisión).
- **Buscaminas** El jugador selecciona una celda dentro de una cuadrícula, evitando seleccionar una que contenga minas. Cada celda muestra la cantidad de minas adyacentes a la celda seleccionada. La implementación mínima para esto debe ser de una cuadrícula de 6x6 celdas. Se espera una estructura basada en clases para representar el tablero, las celdas y la lógica de juego (abrir, marcar, descubrir adyacentes). Deben utilizar DP para resolver de forma eficiente la propagación de celdas vacías, almacenar combinaciones ya evaluadas y actualizar el estado del tablero.
- **Space Invaders** La idea es que se implemente el nivel 1 de este juego. En este, el jugador controla una nave que se mueve horizontalmente y que dispara proyectiles para destruir oleadas de enemigos que se desplazan en grupo de manera vertical, de tal manera que el juego se acaba cuando los enemigos llegan a la altura donde está la nave controlada por el jugador, o el jugador es capaz de destruir a todos los enemigos. Se deben implementar como mínimo las clases para la nave, los enemigos, los proyectiles y la interfaz del juego, con jerarquías que permitan reutilizar comportamientos comunes (por ejemplo, movimiento o colisión).
- **Pac Man** EL jugador controla a Pacman a través de una cuadrícula en forma de laberinto comiendo 'puntos' y evitando ser atrapado por fantasmas. Si estos fantasmas tocan a Pacman este pierde una vida, con máximo de 3 vidas. Se debe implementar un laberinto con al menos una zona en la cual pacman puede moverse a la derecha y 'aparecer' a la izquierda y viceversa. La forma del laberinto puede ser fija. De esta manera, se debe programar las entidades principales: Pac-man, laberinto, fantasmas, puntos en forma de clases con métodos que gestionen el movimiento, colisiones y cambios de estado.
- **Connect 4** Dos jugadores alternan turnos para dejar fichas en una cuadrícula vertical con el objetivo de alinear cuatro del mismo color. De esta manera, se debe implementar como mínimo las clases tablero, jugadores y fichas, manejando lógica de inserción, turnos y detección de victoria. Los estados del tablero pueden ser almacenados a través de DP de tal manera que se optimicen la toma de decisiones.

Entrega del proyecto

La entrega del proyecto consiste en un reporte y una presentación. La extensión máxima de la presentación es de 10 minutos. La estructura de ambas debe consistir de los siguientes puntos:

- **Introducción:** Breve descripción del juego.
- **Metodología:** Se expliquen las diferentes acciones dentro del juego, los posibles estados y la implementación desarrollada para lograr realizar una simulación del juego.
- **Resultados:** Se muestran los resultados del rendimiento del agente ya sea mediante gráficos de recompensa promedio por episodio, puntaje propio del juego, o tasa de aciertos. Además se debe comparar el rendimiento del agente desarrollado con dos casos base: **1.** Un jugador base que toma decisiones aleatorias para cada acción de tiempo del juego. **2.** Un jugador humano (usted) el cual toma decisiones racionalmente para cada situación del juego.
- **Análisis y discusión:** Se debe comentar respecto al rendimiento del agente, la validez de los supuestos realizados y el resultado de las decisiones de diseño efectuadas. Se debe concluir respecto al rendimiento del agente frente a las *baselines* y como se podría mejorar aún más lo conseguido.